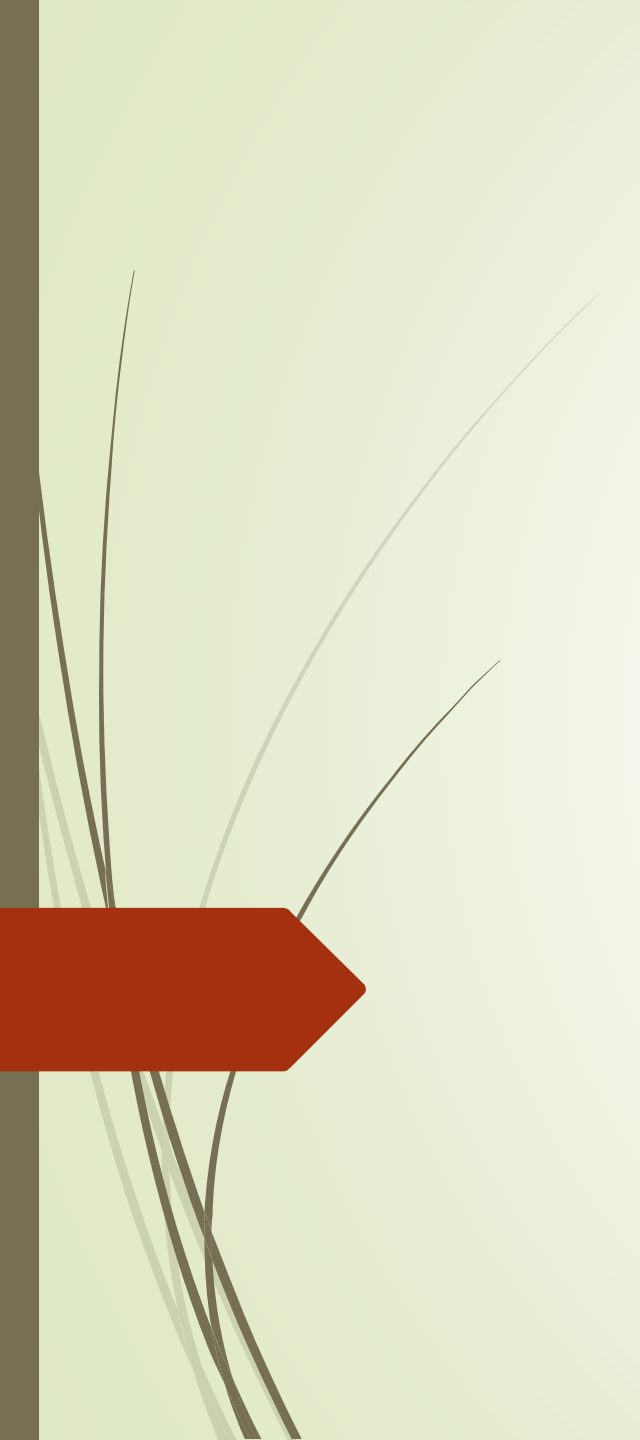




Algorithmique & Structures de Données 1 (ASD-1)

ING-3 / Cours intégré (42h : 21h Cours, 21h TD)

Hajer SALHI

Chapitre 2 : Les procédures et les fonctions

Plan du cours

Introduction & Motivations

Chapitre 1 : Les Notions de base de l'algorithmique

Chapitre 2 : Les procédures et les fonctions

1. Introduction
2. Les paramètres
3. Définition et Utilisation
4. Passage de paramètres

Chapitre 3 : Quelques algorithmes sur les Tableaux

Chapitre 4 : La récursivité

Chapitre 5 : Les Structures de données dynamiques

Chapitre 6 : Les Structures de données abstraites

Introduction

La résolution d'un problème complexe peut engendrer des milliers de lignes de code.

- ➔ Algorithme long,
- ➔ difficile à écrire, à interpréter, et à maintenir

Solution : Méthodologie de résolution structurée

La programmation structurée

Idée : Partitionner le problème en des sous problèmes moins complexes
Partitionner la résolution en des sous algorithmes (modules)

Avantages :

- Clarté de l'algorithme
- Lisibilité de la lecture d'un algorithme
- Facilité de maintenance : éventuelles corrections ou mises à jour
- Réutilisation des sous algorithmes

Introduction

Appelant : tout algorithme, sous algorithme (procédure ou fonction) qui utilise (fait appel) à un autre sous algorithme

Appelé : le sous algorithme (procédure ou fonction) utilisé (appelé) dans un autre sous algorithme

2 types de sous algorithmes :

- ➡ Procédure
- ➡ Fonction

Différence entre procédure et les fonction

Fonction :

- Une fonction réalise un traitement dont le résultat peut être, par la suite, utilisé par une instruction.
 - ➔ Valeur de retour
- Une fonction peut être vue comme un opérateur produisant une valeur.

Procédure :

- Une procédure est une instruction composée qui peut prendre des paramètres et dont le rôle est de modifier l'état courant.
- Les procédures ne retournent pas de résultat.

7

Les paramètres

Les paramètres



Les sous algorithmes communiquent entre eux à travers des objets appelés paramètres.

On distingue 3 types de paramètres :

- **Les paramètres données** : représentant les informations dont a besoin le sous algorithme pour réaliser son traitement. Ce sont les entrées de la fonction.
- **Les paramètres résultats** : représentant les résultats calculés par le sous algorithme. Ce sont les sorties de la fonction.
- **Les paramètres données/résultats** : représentant à l'appel des données qui seront transformés par le sous algorithme en résultats.

Les paramètres : formels et effectifs

- **paramètres formels** : les objets utilisés pour la description d'un sous algorithme (procédure ou fonction)
- **paramètres effectifs** les objets utilisés lors de son appel.
- Un paramètre formel est toujours une variable.
- Un paramètre effectif peut être :
 - une variable
 - une constante
 - une expression arithmétique
 - un appel de fonction

Les paramètres : formels et effectifs

- Pour tout **paramètre formel** on fait correspondre un **paramètre effectif**.
- Le paramètre formel et le paramètre effectif correspondant doivent avoir le **même type ou être de types compatibles**.
- La correspondance entre paramètres formels et paramètres effectifs se fait selon **l'ordre de leurs apparitions** dans la définition et dans l'utilisation de la procédure ou la fonction.

Définition et Utilisation

Définition et Utilisation

Syntaxe de Définition d'une procédure

Pour définir une procédure on adoptera la syntaxe suivante :

```
<Nom_proc> (<liste_param_formels>)  
Var <declaration_var_locales>  
Debut  
    <Corps_procedure>  
Fin
```

Où :

- **Nom_proc** : désigne le nom de la procédure.
- **Liste_param_formels** : la liste des paramètres formels définis par leurs identificateurs et leurs types et séparés par des points virgules (;). On peut regrouper plusieurs paramètres ayant le même type en une seule déclaration et en utilisant comme séparateur la virgule (,)

Un paramètre **résultat ou donnée/résultat** doit être **précédé par le mot clé var.**

- **declaration_var_locales** : la liste des variables (autres que les paramètres) utilisées dans la procédure et définies selon la même syntaxe que pour les variables d'un algorithme.
- **Corps_procedure** : la suite des instructions décrivant le traitement à effectuer.

Définition et Utilisation

Exemple 1 :

Ecrire une procédure qui permet de lire N valeurs entières et de calculer la plus petite et la plus grande parmi ces N valeurs. Les entiers saisis doivent être strictement supérieurs à 0 et strictement inférieurs à 100.

Min_Max (N : entier: var min, max : entier)

VAR i, x : entier

Début

min $\leftarrow$ 100

max $\leftarrow$ 0

pour i de 1 à N **faire**

Répéter

Lire(x)

jusqu'à (x > 0 et x < 100)

Si x < min **alors**

min $\leftarrow$ x

Fin Si

Si x > max **alors**

max $\leftarrow$ x

Fin Si

Finpour

Fin

- Le nom de cette procédure est **Min_Max**
- Les paramètres formels sont : l'entier N comme paramètre donné, les entiers min et max comme paramètres résultats (précédés par le mot clé var).
- Nous avons utilisé 2 variables locales de type entier : i et x

Définition et Utilisation

Syntaxe de Définition d'une fonction

La définition d'une fonction diffère légèrement de celle de la procédure. En effet, la différence se situe au niveau de l'entête et dans le corps réservé au traitement:

<Nom_fonc> (<liste_param_formels>) : <Type_fonc>

Var <declaration_var_locales>

Debut

<Corps_fonction>

Fin

Où :

- **<Nom_fonc>**, **<Liste_param_formels>**, **<declaration_var_locales>** : définissent les mêmes concepts que pour la procédure
- **<Type_fonc>** : il s'agit du type de la fonction **qui devrait être un type scalaire (simple)**. C'est l'un des résultats calculés par la fonction est porté par elle-même. De ce fait, un type est associé à la fonction caractérisant ainsi ce résultat.
- **<Corps_fonction>** : cette partie doit obligatoirement contenir, en plus des instructions décrivant le traitement à effectuer, une instruction d'affectation du résultat que devrait porter la fonction au nom de la fonction elle-même.

Définition et Utilisation

Exemple 2 :

Nous reprenons le problème de l'exemple 1 décrit sous la forme d'une fonction :

Min_Max (N : entier: var min : entier) : **entier**

VAR i, x, max : entier

Début

min $\leftarrow$ 100

max $\leftarrow$ 0

pour i de 1 à N **faire**

Répéter

Lire(N)

jusqu'à (x > 0 et x < 100)

Si x < min **alors**

min $\leftarrow$ x

Fin Si

Si x > max **alors**

max $\leftarrow$ x

Fin Si

Finpour

Min_Max $\leftarrow$ max // Retourner (max)

Fin

- Dans cette fonction, nous avons choisi de définir le résultat valeur minimale par un paramètre formel (min) et de faire porter le deuxième résultat (valeur maximale) par la fonction elle-même.

Définition et Utilisation

Syntaxe d'Utilisation

Lors de l'appel d'un sous algorithme (procédure ou fonction) à partir d'un algorithme ou sous algorithme appelant :

<Nom> (<liste_param_effectifs>)

<Nom> : est le nom de la procédure ou la fonction –

<liste_param_effectifs>:

- séparés par des virgules (',').
 - Le nombre de ces paramètres = nombre des paramètres formels.
 - compatibilité des types entre paramètres formels et paramètres effectifs.
 - Respecter l'ordre d'apparition.
-
- Une procédure est utilisée comme une instruction.
 - Une fonction est utilisée comme une variable.
 - Autrement dit, la fonction est utilisée dans toute instruction exception faite de la partie gauche de l'affectation et l'instruction de lecture.

Définition et Utilisation

Exemple 3 : Version 1

On souhaite écrire un algorithme qui lit un entier N supérieur à 3, puis saisit N valeurs entières et affiche la plus petite et la plus grande parmi ces N valeurs. Les entiers saisis doivent être strictement supérieurs à 0 et strictement inférieurs à 100.

```
Lire_Nombre() : entier  
VAR      N : entier  
Début  
    Répéter  
        Lire(N)  
    jusqu'à ( N > 3)  
  
    Lire_Nombre ← N      // Retourner (N)  
Fin
```

```
Algorithme Affiche_Min_Max  
VAR      N, min, max : entier  
Début  
    N ← Lire_Nombre ()  
    Min_Max(N, min, max)  
    Ecrire( 'La plus grande valeur est :', max)  
    Ecrire( 'La plus petite valeur est :', min)  
Fin
```

Dans cet algorithme, on a utilisé :

- Une fonction lire_nombre pour la saisie de l'entier N. Cette fonction est sans paramètre.
- La procédure Min_Max de l'exemple 1.

Définition et Utilisation

Exemple 3 : Version 2

On souhaite écrire un algorithme qui lit un entier N supérieur à 3, puis saisit N valeurs entières et affiche la plus petite et la plus grande parmi ces N valeurs. Les entiers saisis doivent être strictement supérieurs à 0 et strictement inférieurs à 100.

Lire_Nombre (var N :entier)

Début

Répéter

Lire(N)

jusqu'à (N > 3)

Fin

Algorithme Affiche_Min_Max

VAR N, min, max : entier

Début

Lire_Nombre (N)

Min_Max(N, min, max)

Ecrire('La plus grande valeur est :', max)

Ecrire('La plus petite valeur est :', min)

Fin

Dans cet algorithme, nous avons utilisé une procédure à la place d'une fonction pour la saisie de l'entier N.

Définition et Utilisation

Exemple 3 : Version 3

On souhaite écrire un algorithme qui lit un entier N supérieur à 3, puis saisit N valeurs entières et affiche la plus petite et la plus grande parmi ces N valeurs. Les entiers saisis doivent être strictement supérieurs à 0 et strictement inférieurs à 100.

Lire_Nombre (**var** N :entier)

VAR N : entier

Début

Répéter

 Lire(N)

jusqu'à (N > 3)

Fin

Algorithme Affiche_Min_Max

VAR N, min, max : entier

Début

 Lire_Nombre (N)

 max ← Min_Max(N, min)

 Ecrire('La plus grande valeur est :', max)

 Ecrire('La plus petite valeur est :', min)

Fin

Dans cet algorithme, nous avons utilisé la fonction Min_Max de l'exemple 2.

Passage de paramètres

Passage de paramètres

Deux modes de passage de paramètres:

- **Le passage par valeur** : **Valeur** du paramètre effectif transmise au paramètre formel.
 - tout changement effectué sur le paramètre formel **n'affectera en aucun cas** le paramètre effectif.
 - principalement utilisé sur **les paramètres données** qui ne seront pas modifiés au niveau du sous algorithme appelé.
- **Le passage par adresse** : **Adresse** du paramètre effectif transmise au paramètre formel correspondant.
 - tout changement effectué sur le paramètre formel **affectera automatiquement** le paramètre effectif.
 - principalement utilisé sur **les paramètres résultats ou données/résultats** afin de récupérer les résultats calculés par le sous algorithme appelé

Passage de paramètres

- Les variables locales n'existent plus après la fin de l'appel de fonction/procédure.
- Lors d'un passage par valeur, il y a une copie des valeurs.
- Lors d'un passage par adresse, il y a une copie des adresses

Passage de paramètres

Exemple 4 :

Soit l'algorithme suivant faisant appel à la procédure ajoute_un :

Algorithme Test

VAR x : entier

Début

x $\leftarrow$ 9

ajout_un (x)

Ecrire('x')

Fin

Autrement dit, son unique paramètre est passé par valeur, alors la valeur de x dans l'algorithme Test ne sera pas altéré et la valeur qui sera affichée est 9.

Si la procédure ajoute_un est définie comme suit :

ajoute_un (a : entier)

VAR N : entier

Début

a $\leftarrow$ a + 1

Fin

Passage de paramètres

Exemple 4 :

Soit l'algorithme suivant faisant appel à la procédure ajoute_un :

```
Algorithme Test  
VAR      x : entier  
Début  
    x ← 9  
    ajout_un (x)  
    Ecrire( 'x')  
Fin
```

Autrement dit, son paramètre est passé par adresse, alors la valeur de x dans l'algorithme Test sera modifiée et la valeur affichée est alors 10.

Or, si la procédure ajoute_un est définie comme suit :

```
ajoute_un (var a : entier)  
VAR      N : entier  
Début  
    a ← a + 1  
Fin
```

Exercices

Exercice 1 :

Écrire une fonction, nommée puissance, qui prend en argument un réel x et un entier n , calcule et retourne la valeur de x^n .

Exercice 2 :

Écrire une procédure heure qui, étant donné un entier m correspondant à une durée exprimée en minutes depuis minuit affiche son équivalent exprimé en heures et minutes. Un message d'erreur sera affiché si la durée est supérieure à une journée (c'est-à-dire > 1440 minutes).

Exercices

Exercice 3 :

Ecrire un algorithme qui lit 3 entiers A , B et C tel que $A \leq B$ et $C \neq 0$, affiche les entiers compris entre A et B qui sont divisibles par C ainsi que leur nombre.

Exemple: Si $A=3$ $B=19$ $C=4$ On affichera

4 8 12 16

Le nombre des entiers divisibles par C est 4

Exercices

Exercice 4 :

Un nombre parfait est un nombre présentant la particularité d'être égal à la somme de tous ses diviseurs excepté lui-même. Le premier nombre parfait est le nombre 6 qui est bien égal à $1+2+3$.

Ecrire une procédure `nombres_parfaits` qui retourne la liste des nombres parfaits inférieurs à un entier `nmax` entré au clavier par l'utilisateur dans le programme principal. Pour ce faire, il peut être utile d'écrire la fonction `somme_div` qui retourne la somme des diviseurs d'un nombre entier passé en paramètre.

Exemple :

Entrez un entier : 3000

Les nombres parfaits inférieurs à 3000 sont :

6 28 496

Exercices

Exercice 5 :

Ecrire un algorithme qui permet de lire deux entiers X et Y strictement positifs, affiche "X et Y sont AMIS" ou "X et Y ne sont pas AMIS".

Deux nombres X et Y sont dits nombres amis si la somme des diviseurs de X est égale à Y et la somme des diviseurs de Y est égale à X .

Par exemple 220 et 284 sont des nombres amis.

Exercices : Corrigé

Exercice 1 :

Écrire une fonction, nommée puissance, qui prend en argument un réel x et un entier n , calcule et retourne la valeur de x^n .

```
Fonction puissance (x: réel, n: entier) : réel
VAR    i : entier
Début
    p ← 1
    pour i de 1 à n faire
        p ← p*x
    Fin pour
    puissance ← p      // Retourner (p)
FIN
```

Exercices : Corrigé

Exercice 2 :

Ecrire une procédure heure qui, étant donné un entier m correspondant à une durée exprimée en minutes depuis minuit affiche son équivalent exprimé en heures et minutes. Un message d'erreur sera affiché si la durée est supérieure à une journée (c'est-à-dire > 1440 minutes).

```
Procédure heure ( m : entier )  
VAR  
Début  
    Si m > 1440 alors  
        Ecrire('heure invalide')  
    Sinon  
        Ecrire ( m DIV 60,'h', m MOD 60, 'min' )  
    Finsi  
FIN
```

Exercices : Corrigé

Exercice 3 :

Ecrire un algorithme qui lit 3 entiers A, B et C tel que $A \leq B$ et $C \neq 0$, affiche les entiers compris entre A et B qui sont divisibles par C ainsi que leur nombre.

Exemple: Si A=3 B=19 C=4 On affichera

4 8 12 16

Le nombre des entiers divisibles par C est 4

Procédure Saisie(**var** z, x, y : entier)

Début

 Répéter

 Lire (x, y)

 Jusqu'à ($x \leq y$)

 Répéter

 Lire (z)

 Jusqu'à ($z \neq 0$)

Fin

Fonction Div(A,B, C :entier) : entier

VAR nb, i : entier

Début

 nb ← 0

 pour i de A à B faire

 Si $i \bmod C = 0$ alors

 Ecrire (i)

 nb ← nb + 1

 Finsi

 Finpour

 Retourner (nb) // Div ← nb

Fin

Algorithme

VAR A, B, C , nb : entier

Début

 Saisie (C, A, B)

 nb ← Div (A, B,C)

 Ecrire ('Le nombre des entiers divisibles par', C , 'est' nb)

FIN

Exercices d'applications

Série d'exercices N°3 :

- **Thème principal de la série : Les procédures et les fonctions**